Übungsblatt 3: Einführung FreeRTOS und Präemption

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Zielsetzung

Ein FreeRTOS-Kernel läuft auf dem Cortex-M4 des Boards **NUCLEO-H755ZI-Q** (MCU STM32H755ZIT6). Der CM7 bleibt bei der vom Cube-Projekt vorgesehenen Startaufgabe und erhält keinen zweiten Scheduler. Sie legen drei CMSIS-RTOS2-Threads an, trennen die Zeitbasen in dieser Laborvariante und prüfen Präemption sowie Starvation. Es liegt kein fertiges `.ioc`-Projekt im Kursrepository; Menüs gelten für die bei Ihnen installierte Cube-Version.

Aufgabe 1: Toolchain

1. Projekt für das Board **NUCLEO-H755ZI-Q**, FreeRTOS nur auf dem CM4, Interface **CMSIS_V2**. Ab CubeIDE 2.0 oft: STM32CubeMX separat, danach Import. Ältere IDE: integriertes Device-Configuration-Tool. Die tatsächlichen Menünamen an der Version prüfen.
2. `configTOTAL_HEAP_SIZE` so setzen, dass die drei Threads, Idle und gegebenenfalls der Timer-Thread in die Map passen. 16 KB ist ein Startwert, kein Nachweis.
3. Für **dieses** Labor die HAL-Zeitbasis auf einen freien Timer legen, üblicherweise **TIM6**, und SysTick dem Kernel lassen. Ein gemeinsamer SysTick ist bei korrekter Integration möglich; die Trennung ist die gewählte Konfiguration, kein universelles Absturzgesetz. Handler, Priorität und Frequenz prüfen.
4. Lesen Sie in `cmsis_os2.c` nach, wie `osPriority_t` auf native Prioritäten abgebildet wird, und ob `configMAX_PRIORITIES` dafür reicht.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

SysTick bedient `configTICK_RATE_HZ`. TIM6 bedient `HAL_IncTick`, sofern die SYS-Timebase so generiert wurde. Beide IRQs liegen auf dem CM4. Stehen beide auf demselben Vektor ohne abgestimmte Kette, überschreiben sich die Handler. Das ist ein Integrationsfehler, keine Eigenschaft „SysTick darf nie die HAL sehen“.

Notieren Sie Kernelversion, `configTICK_RATE_HZ`, `configMAX_PRIORITIES` und die Bibliotheksschwelle `configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY`. Die verschobene `configMAX_SYSCALL_INTERRUPT_PRIORITY` ist der Registerwert, nicht die logische Stufe.

Aufgabe 2: Drei Threads

Legen Sie an, generieren Sie, und prüfen Sie Handles:

Thread	CMSIS-Priorität	Stack
Emergency	`osPriorityHigh`	1024 Byte, Feld gegen generierten Wert prüfen
Heartbeat	`osPriorityNormal`	512 Byte
HeavyMath	`osPriorityBelowNormal`	1024 Byte

`osKernelInitialize` vor `osThreadNew`. `osKernelStart` bei Erfolg kehrt normalerweise nicht zurück; der Zweig dahinter bleibt für den Fehlerfall. Funktionen nicht in `USER CODE BEGIN Header_…` schreiben.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

Die generierten Namen können `StartHeartbeatTask` heißen und in `main.c` oder `freertos.c` liegen. Reihenfolge der Dringlichkeit: Emergency, Heartbeat, HeavyMath, Idle. Eine leere High-Ready-Schleife in Emergency würde die anderen aushungern. Emergency wartet auf ein Thread-Flag (Aufgabe 3).

Aufgabe 3: Heartbeat, HeavyMath, Emergency

LED **LD2 / PE1** (MB1363) dem CM4 zuweisen. Taster und EXTI am Board prüfen; den IRQ erst freigeben, wenn das Emergency-Handle existiert und der Kernel läuft. Vorhandenen HAL-Callback nutzen, keine zweite Handler-Definition.

1. Heartbeat: Toggle alle 500 ms **Sollraster** mit `osDelayUntil`. Ein voller On-Off-Zyklus dauert damit etwa 1 s. Nicht `HAL_Delay`.
2. HeavyMath: dauerhaft Ready, kein Delay, IRQs nicht sperren. Das Rechenergebnis einer `volatile`-Variablen zuweisen, damit der Compiler die Schleife behält.
3. Emergency: `osThreadFlagsWait` auf Bit 0, kurze Reaktion (eigener Messpin, nicht die Heartbeat-LED), wieder warten. ISR: `osThreadFlagsSet`. Rückgabewert über die Fehlerbits prüfen, nicht mit `osOK` vergleichen.
4. Erklären Sie, warum HeavyMath die CPU fordern kann und die LED trotzdem umschaltet.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

```c
static uint32_t ms_to_ticks_ceil(uint32_t ms)
{
    uint32_t f = osKernelGetTickFreq();
    if (ms == 0u || f == 0u) return 0u;
    return (uint32_t)(((uint64_t)ms * f + 999u) / 1000u);
}

/* USER CODE BEGIN StartHeartbeatTask */
uint32_t next = osKernelGetTickCount();
uint32_t period = ms_to_ticks_ceil(500);
for (;;) {
    HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
    next += period;
    if (osDelayUntil(next) == osErrorParameter)
        next = osKernelGetTickCount();
}
/* USER CODE END StartHeartbeatTask */

/* USER CODE BEGIN StartHeavyMathTask */
static volatile uint32_t acc;
for (;;) {
    acc = acc * 1664525u + 1013904223u;
}
/* USER CODE END StartHeavyMathTask */

/* USER CODE BEGIN StartEmergencyTask */
for (;;) {
    uint32_t fl = osThreadFlagsWait(0x1u, osFlagsWaitAny, osWaitForever);
    if ((fl & osFlagsError) != 0u) continue;
    /* kurze Reaktion, dann wieder warten */
}
/* USER CODE END StartEmergencyTask */

/* im bestehenden EXTI-Callback, nach laufendem Kernel */
osThreadFlagsSet(EmergencyHandle, 0x1u);
```

`EmergencyHandle` durch den generierten Namen ersetzen. Die IRQ-Priorität muss auf der erlaubten Seite der Syscall-Schwelle liegen.

Präemption: HeavyMath ist Ready und rechnet. Heartbeat hat die höhere Priorität, wird zum Sollpunkt Ready und verdrängt HeavyMath über PendSV. Danach blockiert Heartbeat wieder. Das ist kein Fristbeweis und kein „Jitter 0“. Dieselbe Flag-Flanke kann mehrere Tasterereignisse zusammenfassen.

`HAL_Delay` wäre Busy-Wait im Zustand Running und trotzdem von Emergency präemptierbar. Für diese Wartezeiten ist er hier falsch, weil Heartbeat die CPU nicht freigeben würde.

Aufgabe 4: Starvation

1. Setzen Sie HeavyMath auf `osPriorityAboveNormal` (über Heartbeat, unter Emergency). Nicht beide auf `osPriorityHigh`, sonst entsteht ein Gleichstand; den nur untersuchen, wenn `configUSE_TIME_SLICING` bekannt ist.
2. Beobachten Sie PE1. Eine bereits leuchtende LED muss nicht dunkel werden; sie kann einfach an bleiben.
3. Ist das Priority Inversion? Hilft `osThreadYield` in HeavyMath?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4

Heartbeat wird Ready und bleibt unausgeführt, solange HeavyMath dauerhaft Ready und höher ist. Das ist Starvation. Priority Inversion bräuchte eine Ressource, die eine niedrige Aufgabe hält und auf die eine hohe wartet. `osThreadYield` wählt bei strikter Priorität wieder HeavyMath. Anschließend die Prioritäten zurücksetzen: HeavyMath `BelowNormal`, Heartbeat `Normal`, Emergency `High`.

Diagnose, wenn nichts blinkt: Handle/`osKernelStart`, Takt und EXTI, Verdrängung, Assert, Overflow-Hook, Watermark, Heap und Map. Ein Stacküberlauf faultet nicht zuverlässig. Die Stackgröße nicht pauschal verdoppeln.